SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1735**

Name: Y.Vyshnavi Reddy(192412642)

Faculty:Thalapathi Rajasekaran R

Code of Conduct:

I Y.Vyshnavi Reddy(192412642) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Y.Vyshnavi Reddy

DETAILED ANSWERS :

Answer 1. Hospital Shift Scheduling — Backtracking Search

1. Variables and domains

Let X1, X2 and X3 represent the shifts assigned to doctors D1, D2 and D3. Each variable initially has domain {Morning (M), Afternoon (A), Night (N)}.

2. Constraints

- D1 ≠ N.
- D3 ≠ M.
- D2 must be earlier than D3: shift(D2) < shift(D3), where M < A < N.
- All shifts are different, and every doctor receives exactly one shift.

3. Backtracking and constraint checks

Step	Assignment considered	Constraint check	Decision
1	D2 = M	M is a valid shift; no conflict yet.	Accept temporarily.
2	D3 = A	D2 is before D3 (M < A); D3 is not M.	Accept temporarily.
3	D1 = N	Violates D1 cannot work Night; also N is already not assigned uniquely if D3 later changes.	Reject; backtrack.
4	D1 = A	A is already assigned to D3, violating one doctor per shift.	Reject.
5	D1 = N after choosing D3 = N is not possible because D2 must be before D3 and D1 cannot be N.	Reject branch.	
6	D2 = M, D3 = N, D1 = A	D2 before D3; D3 not M; D1 not N; all shifts distinct.	Accept solution.

4. Final valid schedule

Shift	Doctor
Morning	D2
Afternoon	D1
Night	D3

Conclusion: Backtracking builds an assignment incrementally and rejects a partial assignment as soon as a constraint is violated. The schedule above satisfies all five constraints.

Answer 2. Robot Navigation — Manhattan Heuristic and Best-First Search

Important note: The supplied assessment text does not include a grid drawing or the coordinates of S, G, and obstacles. To demonstrate the required process, assume S=(1,1), G=(5,5), and obstacles at (2,2), (3,2), and (4,4). Coordinates are (row, column). This example must be replaced if your instructor provided a separate grid.

Manhattan heuristic: $h(r,c) = |r - rG| + |c - cG|$. Each legal move costs 1. The robot cannot enter an obstacle. Greedy Best-First orders nodes by h; however, Greedy Best-First alone does not guarantee an optimal path. To establish optimality, use A* priority $f(n)=g(n)+h(n)$, which uses the same Manhattan heuristic.

Illustrative grid (X = obstacle):

	Col 1	**Col 2**	**Col 3**	**Col 4**	**Col 5**
Row 1	S	·	·	·	·
Row 2	·	X	·	·	·
Row 3	·	X	·	·	·
Row 4	·	·	·	X	·
Row 5	·	·	·	·	G

One optimal path is: (1,1) → (1,2) → (1,3) → (1,4) → (1,5) → (2,5) → (3,5) → (4,5) → (5,5). Its cost is 8 because it uses eight unit-cost moves. The Manhattan lower bound at S is $|1-5|+|1-5|=8$, so no route can cost less than 8; the route achieves that lower bound and is therefore optimal.

Example priority/expansion trace using A* (ties are broken by preferring the smaller row, then smaller column):

Step	**Node expanded**	**g**	**h**	**f=g+h**	**Key queue update**
0	S=(1,1)	0	8	8	Add (1,2) and (2,1), each with g=1 and f=8.
1	(1,2)	1	7	8	Add (1,3), g=2, f=8; reject obstacle (2,2).
2	(1,3)	2	6	8	Add (1,4), g=3, f=8; add (2,3), g=3, f=9.
3	(1,4)	3	5	8	Add (1,5), g=4, f=8; add (2,4), g=4, f=9.
4	(1,5)	4	4	8	Add (2,5), g=5, f=8.
5	(2,5)	5	3	8	Add (3,5), g=6, f=8.
6	(3,5)	6	2	8	Add (4,5), g=7, f=8.
7	(4,5)	7	1	8	Add G=(5,5), g=8, f=8.
8	G=(5,5)	8	0	8	Goal reached; terminate.

Conclusion: For the illustrative map, the optimal path cost is 8. The Manhattan heuristic is admissible for a four-direction grid with unit movement costs because it never exceeds the number of moves still required. If the intended grid has different obstacle locations, its expansion sequence and path may differ.

Answer 3. Autonomous Rescue Robot — Online Search

a) Constraint analysis

Constraint	Effect on decisions
Four-direction movement	Only adjacent vertical/horizontal cells are legal; diagonal movement is prohibited.
Blocked cells	The robot must remove known blocked cells from its legal successor list.
Limited sensing	Only adjacent cells can be observed; the complete map is not initially available.
Movement cost 1	Every normal move adds 1 to the accumulated path cost g.
Risky-zone surcharge 2	Entering a risky cell costs 1+2=3, so the robot should compare safer alternatives using total cost.
Dynamic knowledge	After each observation, the robot updates its map and replans if a newly discovered obstacle or risk changes the best route.
Safety requirement	A route should be feasible and avoid blocked cells; among safe feasible routes, minimise total cost.

b) Proposed solution: Online Search Agent with Uniform Cost Search (UCS) on the currently known map

1. Initialise the current position as S. Record only known information; unknown adjacent cells are treated as unobserved, not automatically safe.
2. Sense the four adjacent cells and label each as free, blocked, or risky if the sensor can identify risk.
3. For known reachable cells, use step cost c=1 for an ordinary cell and c=3 for entering a risky zone. UCS prioritises the frontier node with the smallest accumulated g-cost.
4. Move toward the lowest-cost frontier cell. On arrival, sense adjacent cells and update the internal map.
5. If a newly discovered obstacle invalidates the planned route, remove that edge/cell and run UCS again from the current position.
6. Stop when the robot reaches G. Report the path and sum all move and risk costs.

Illustrative cost calculation: a route with 6 ordinary moves costs 6. A route with 4 ordinary moves and 1 risky-cell entry costs 4+3=7. The six-move safe route is preferred. Actual route selection requires a map of the cells and their risk labels, which the question does not provide.

c) AI concepts

- Intelligent agent: the robot perceives adjacent cells, maintains a map/belief about the building, selects an action, and acts to reach the survivor.
- Rationality: for the information available at each step, it selects actions expected to minimise total cost while respecting safety constraints. Rationality does not mean it knows hidden obstacles in advance.
- Environment type: partially observable, sequential, dynamic in the sense that knowledge changes as it explores, discrete grid-based, and stochastic/uncertain if obstacle or risk information is not fully known.

d) Justification

UCS is appropriate when edge costs differ, because risky-zone entry costs more than an ordinary move and UCS expands the lowest accumulated-cost frontier first. The online agent component is needed because the map is incomplete and observations arrive during movement. A* could be used if a reliable admissible heuristic is available, but UCS is simpler when no map-based heuristic is specified.

Answer 4. South Indian State Map Colouring — CSP, Degree, MRV and Forward Checking

i) CSP formulation

Variables: TN, KL, KA, AP, TG, GA, MH. Domain for each variable: {Red, Green, Blue}. For every neighbouring pair, the constraint is that the two colours must differ.

Constraint graph (edges are neighbouring pairs):

TN—KL, TN—KA, TN—AP, KL—KA, KA—AP, KA—TG, KA—GA, KA—MH, AP—TG, GA—MH, MH—TG.

ii) Variable ordering

Degree heuristic: KA has six neighbours (TN, KL, AP, TG, GA and MH) and is selected first because it constrains the largest number of other variables. After assigning KA, use MRV: choose an unassigned variable with the smallest remaining domain; break ties by higher degree, then a fixed order for reproducibility.

iii) Backtracking with forward checking

Stage	Assignment	Domains after forward checking
Initial	None	TN, KL, KA, AP, TG, GA, MH: {R,G,B}
1	KA=Red	TN, KL, AP, TG, GA, MH: {Green, Blue}
2	TN=Green (MRV tie broken by degree)	KL={Blue}; AP={Blue}; TG={Green,Blue}; GA={Green,Blue}; MH={Green,Blue}
3	KL=Blue (singleton domain)	AP={Blue}; TG={Green,Blue}; GA={Green,Blue}; MH={Green,Blue}
4	AP=Blue (singleton domain)	TG={Green}; GA={Green,Blue}; MH={Green,Blue}; because AP–TG, remove Blue from TG
5	TG=Green (singleton after checking AP–TG)	GA={Green,Blue}; MH={Blue} because MH–TG and KA=Red
6	MH=Blue	GA={Green} because GA–MH; GA=Green completes assignment.

iv) Final colouring

State	Colour
Tamil Nadu (TN)	Green
Kerala (KL)	Blue
Karnataka (KA)	Red
Andhra Pradesh (AP)	Blue
Telangana (TG)	Green

Goa (GA)	Green
Maharashtra (MH)	Blue

All listed neighbouring pairs have different colours. Degree chooses the most constraining variable first, while MRV prioritises forced assignments. Forward checking removes colours immediately after each assignment, detecting a dead end early and reducing unnecessary backtracking.

Answer 5. 4×4 Sudoku — Alldiff and AC-3

i) Variables and constraints

Each cell Xrc (row r, column c) is a variable with domain {1,2,3,4}; given cells have singleton domains. Each row, column and 2×2 box must satisfy Alldiff: all four values are different.

ii) Initial domains after removing values conflicting with the given clues

Cell	Initial domain
R1C1	{1}
R1C3	{1,3}
R1C4	{3,4}
R2C2	{1,4}
R2C3	{1}
R3C1	{1,2}
R3C2	{1,3}
R3C4	{3}
R4C2	{3}
R4C3	{2,3}

Given clues: R1C2=2, R2C1=3, R2C4=2, R3C3=4, R4C1=4, R4C4=1. The domains above are based on eliminating values already present in the same row, column, or 2×2 box.

iii) AC-3 / constraint propagation

Singleton domains immediately force R2C3=1 and R4C2=3. Then row/column/box propagation gives R1C1=1, R1C3=3, R1C4=4; R2C2=4; R3C1=2, R3C2=1, R3C4=3; R4C3=2. Each singleton is removed from the domains of peer cells. (AC-3 enforces arc consistency on the binary “not equal” constraints corresponding to Alldiff; practical implementations may also use stronger all-different propagation.)

iv) Completed grid

	C1	C2	C3	C4
R1	1	2	3	4
R2	3	4	1	2
R3	2	1	4	3
R4	4	3	2	1

Every row, column and 2×2 box contains 1, 2, 3 and 4 exactly once. Constraint propagation is sufficient for this instance; no backtracking is required once the forced values are propagated.

Answer 6. Cryptarithmetic — TWO + TWO = FOUR

i) Variables and domains

Letter variables: T,W,O,F,U,R, each in {0,…,9}. All letters must have distinct digits. Leading letters T and F cannot be zero. Carry variables C1, C2, C3 each have domain {0,1}; C1 is from units to tens, C2 from tens to hundreds, and C3 from hundreds to thousands.

ii) Column equations

- Units: O + O = R + 10C1, i.e. 2O = R + 10C1.
- Tens: W + W + C1 = U + 10C2, i.e. 2W + C1 = U + 10C2.
- Hundreds: T + T + C2 = O + 10C3, i.e. 2T + C2 = O + 10C3.
- Thousands: C3 = F.

iii) Constraint hypergraph

The units constraint connects {O,R,C1}; the tens constraint connects {W,C1,U,C2}; the hundreds constraint connects {T,C2,O,C3}; the thousands constraint connects {C3,F}. A global AllDifferent constraint connects all six letter variables {T,W,O,F,U,R}.

iv) Step-by-step assignment

7. Try T=7 and check the hundreds column: 2T+C2=14+C2 must end in O and produce a carry C3=1. Taking C2=0 gives O=4 and C3=1.
8. Units column: 2O=8, so R=8 and C1=0. R differs from T and O.
9. Tens column: 2W+C1=U+10C2. With C1=0 and C2=0, choose W=3, giving U=6. The digits are distinct.
10. Thousands column: F=C3=1. F is nonzero and distinct from all other letters.

Valid assignment: T=7, W=3, O=4, F=1, U=6, R=8.

Verification: TWO = 734; FOUR = 1468; 734 + 734 = 1468. All six letters represent different digits, and T and F are nonzero.

Answer 7. Ambulance Routing — A* Search

A* uses f(n)=g(n)+h(n). The road B–D is excluded in the flooded case. Treat roads as bidirectional, as implied by the road-pair description. Ties are broken consistently by lower f, then lower h.

i) Flooded network: open/closed list trace

Step	Node selected (g,h,f)	Open list after expansion (node: g,h,f)	Closed list
0	S (0,6,6)	A:3,6,9; B:2,4,6	S
1	B (2,4,6)	C:3,4,7; A:3,6,9	S, B
2	C (3,4,7)	D:5,2,7; E:6,3,9; A:3,6,9	S, B, C
3	D (5,2,7)	G:7,0,7; E:6,3,9; A:3,6,9	S, B, C, D
4	G (7,0,7)	E:6,3,9; A:3,6,9	S, B, C, D, G

ii) Route and cost with B–D closed

S → B → C → D → G. Total cost = 2 + 1 + 2 + 2 = 7 minutes.

iii) If B–D is open

The route S → B → D → G costs 2 + 2 + 2 = 6 minutes. A* would return this route because it is cheaper than the flooded-network route.

iv) Admissibility and consistency

Admissibility: compare each h with the actual shortest remaining cost in the flooded network. Shortest costs are G=0, D=2, E=4, C=4, B=5, A=8, S=7. Given h values (6,6,4,4,2,3,0 respectively) do not exceed these true costs; therefore the heuristic is admissible.

Consistency requires $h(u) \le \text{cost}(u,v)+h(v)$ for each edge. For the flooded edges: S–A: $6 \le 3+6$; S–B: $6 \le 2+4$; A–C: $6 \le 4+4$; B–C: $4 \le 1+4$; C–D: $4 \le 2+2$; C–E: $4 \le 3+3$; D–G: $2 \le 2+0$; E–G: $3 \le 4+0$. These all hold in both directions for the listed values, so the heuristic is consistent on the flooded network. The closed B–D edge is not checked for the flooded network; if opened, its consistency condition also holds: $h(B)=4 \le 2+h(D)=4$ and $h(D)=2 \le 2+h(B)=6$.

Answer 8. 4-Queens — Min-Conflicts

i) CSP formulation

Variables Q1,Q2,Q3,Q4 represent the row of the queen in columns 1–4. Each domain is $\{1,2,3,4\}$. Constraints: for any $i \neq j$, $Qi \neq Qj$ (no same row) and $|Qi-Qj| \neq |i-j|$ (no same diagonal).

ii) Initial state

Initial rows = (1,3,4,2). Board (Q marks a queen):

Row \ Col	1	2	3	4
1	Q	·	·	·
2	·	·	·	Q
3	·	Q	·	·
4	·	·	Q	·

Conflict count for each queen: Q1=0, Q2=1 (diagonal conflict with Q3), Q3=1 (diagonal conflict with Q2), Q4=0. Conflict counts are per queen; the single attacking pair is counted for both queens.

iii) Min-Conflicts steps

Row 1 gives 1 conflict, row 2 gives 2, row 3 gives 1, and row 4 gives 2. The tie between rows 1 and 3 is broken by choosing the lowest row number, so move Q2 to row 1.

State	Queen rows (Q1,Q2,Q3,Q4)	Per-queen conflicts	Total pair conflicts
Initial	(1,3,4,2)	(0,1,1,0)	1
After move 1	(1,1,4,2)	(1,1,0,0)	1

Move 2: the leftmost queen in conflict is Q1 in column 1. Candidate row conflict counts are row 1: 1, row 2: 2, row 3: 0, row 4: 2. Move Q1 to row 3.

Final state	Queen rows (Q1,Q2,Q3,Q4)	Per-queen conflicts
After move 2	(3,1,4,2)	(0,0,0,0)

Final board:

Row \ Col	1	2	3	4
1	·	Q	·	·
2	·	·	·	Q
3	Q	·	·	·
4	·	·	Q	·

iv) Final solution and justification

The solution is (3,1,4,2), with no row or diagonal conflicts. Min-Conflicts is effective for large n-Queens problems because it starts from a complete assignment and repeatedly repairs a conflicted variable using local conflict counts, instead of exploring the full state space systematically.

Answer 9. Missionaries and Cannibals — BFS

i) Problem formulation

- State: (M,C,B), where M and C are the numbers of missionaries and cannibals on the left bank; $B \in \{L,R\}$ is the boat side. The right bank has $3-M$ missionaries and $3-C$ cannibals.
- Initial state: (3,3,L). Goal state: (0,0,R).
- Actions: boat carries one or two people: (1M), (2M), (1C), (2C), or (1M,1C). Boat cannot cross empty.
- Transition model: subtract the passengers from left-bank counts when the boat goes L→R; add them when it goes R→L. Counts must remain between 0 and 3.
- Goal test: all three missionaries and all three cannibals are on the right bank, i.e. M=0 and C=0.
- Path cost: 1 per crossing; minimise the number of crossings.

ii) Safety constraints

A state is unsafe if on either bank missionaries are present but fewer in number than cannibals. Thus prune any state where ($M>0$ and $M<C$) or where ($3-M>0$ and $3-M<3-C$). Also prune impossible counts, boat moves with more than two passengers, empty-boat moves, and transitions that take more passengers than are present on the departure bank.

iii) BFS solution

BFS explores all valid states in increasing order of crossings. One shortest sequence is shown below. L→R means left to right; R→L means right to left.

Crossing	**Boat move**	**State after crossing (Mleft,Cleft,boat)**
Start	—	(3,3,L)
1	2 cannibals L→R	(3,1,R)
2	1 cannibal R→L	(3,2,L)
3	2 cannibals L→R	(3,0,R)
4	1 cannibal R→L	(3,1,L)
5	2 missionaries L→R	(1,1,R)
6	1 missionary + 1 cannibal R→L	(2,2,L)
7	2 missionaries L→R	(0,2,R)
8	1 cannibal R→L	(0,3,L)
9	2 cannibals L→R	(0,1,R)
10	1 missionary R→L	(1,1,L)
11	1 missionary + 1 cannibal L→R	(0,0,R)

iv) Minimum crossings and BFS justification

The solution uses 11 crossings. BFS guarantees the minimum number because every crossing has equal cost 1 and BFS explores states by increasing path depth. The first time the goal is reached, no shorter valid sequence can exist.

Answer 10. Sensorless Corridor Robot — Belief-State Search

i) Initial belief state and size

Because the robot has no sensor and may start in any corridor cell, its initial belief state is B0={1,2,3,4}. A belief state is a subset of the four physical states, so the full belief-state space has 2^4=16 subsets (including the empty set mathematically; reachable beliefs depend on the actions and initial belief).

ii) Belief states after [Right, Right, Right]

Action	Belief state	Explanation
Start	{1,2,3,4}	All start positions are possible.
Right	{2,3,4}	1→2, 2→3, 3→4, 4→4.
Right	{3,4}	2→3, 3→4, 4→4.
Right	{4}	3→4 and 4 stays at 4. The robot is now certain it is in cell 4.

iii) Why no shorter sequence guarantees cell 4

After k consecutive Right actions, a possible initial position at cell 1 has moved only to cell min(1+k,4). For k=0,1,2 the possible final positions include cells 1, 2, or 3 respectively, so the belief state is not {4}. Three Right actions map every possible starting cell to cell 4. Any Left action cannot improve the worst-case guarantee compared with moving right toward the right boundary; in particular, from cell 1 the robot needs at least three net rightward moves to reach cell 4. Thus three actions are necessary and sufficient.

iv) With a bump sensor

Contingency plan: repeatedly attempt Left until the bump sensor reports that the robot has hit the left wall. At that point the robot is certainly at cell 1. Then execute Right three times to reach cell 4.

Comparison: the sensorless plan is a fixed sequence Right, Right, Right (exactly 3 actions). The bump-sensor plan is conditional and guarantees localisation at the left boundary before moving right; it can require up to four Left attempts to detect the wall, followed by three Right actions (worst case 7 actions under the stated “reports when it tries to move into a wall” model). It uses feedback and adapts to the unknown start position.